

```
"""
```

```
FGE Isolde Voss Lookbook – Full-Bleed Rebuild  
Dragon Seed framing elegance: thin gold border, corner brackets,  
diagonal multi-line rule clusters, bottom scrim with burned text.
```

```
"""
```

```
from PIL import Image, ImageDraw, ImageFont, ImageFilter  
import math  
import os
```

```
# — CONFIG —————
```

```
OUTPUT_DIR = "/home/claude/lookbook_output"  
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
# Output page size: standard portrait 8.5×11 at 300dpi equivalent  
PAGE_W = 2550  
PAGE_H = 3300
```

```
# Gold color palette (match Dragon Seed card)  
GOLD_BRIGHT = (212, 175, 55) # #D4AF37 warm gold  
GOLD_MID     = (180, 147, 40) # slightly darker mid  
GOLD_DIM     = (140, 112, 28) # dim for secondary elements  
WHITE_DIM    = (220, 220, 220) # off-white for small text  
SCRIM_BLACK  = (0, 0, 0, 200) # semi-transparent scrim
```

```
# Font paths
```

```
FONT_BOLD    = "/usr/share/fonts/truetype/liberation/LiberationSans-Bold.ttf"  
FONT_REG     = "/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf"  
FONT_SERIF_B = "/usr/share/fonts/truetype/liberation/LiberationSerif-Bold.ttf"  
FONT_SERIF_I = "/usr/share/fonts/truetype/liberation/LiberationSerif-Italic.ttf"
```

```
# — HELPERS —————
```

```
def load_font(path, size):  
    try:  
        return ImageFont.truetype(path, size)  
    except:  
        return ImageFont.load_default()
```

```
def fit_cover(img, target_w, target_h):  
    """Scale image to cover target dimensions, center crop."""  
    src_w, src_h = img.size  
    scale = max(target_w / src_w, target_h / src_h)  
    new_w = int(src_w * scale)
```

```

new_h = int(src_h * scale)
img = img.resize((new_w, new_h), Image.LANCZOS)
left = (new_w - target_w) // 2
top = (new_h - target_h) // 2
return img.crop((left, top, left + target_w, top + target_h))

def draw_diagonal_lines_cluster(draw, anchor_x, anchor_y, direction,
                                angle_deg, n_lines=7, spacing=22,
                                length=520, lw=2):
    """
    Draw a cluster of parallel diagonal lines radiating from a corner.
    direction: 'tr' (top-right) or 'bl' (bottom-left)
    angle_deg: angle of lines from horizontal
    """
    angle_rad = math.radians(angle_deg)
    dx = math.cos(angle_rad)
    dy = math.sin(angle_rad)
    # Perpendicular offset direction
    perp_x = -dy
    perp_y = dx

    for i in range(n_lines):
        alpha = i / (n_lines - 1)
        # Fade from bright to dim
        r = int(GOLD_BRIGHT[0] * (1 - alpha * 0.6))
        g = int(GOLD_BRIGHT[1] * (1 - alpha * 0.6))
        b = int(GOLD_BRIGHT[2] * (1 - alpha * 0.6))
        color = (r, g, b)

        offset = i * spacing
        ox = anchor_x + perp_x * offset
        oy = anchor_y + perp_y * offset

        x1 = ox
        y1 = oy
        x2 = ox + dx * length
        y2 = oy + dy * length

        line_w = max(1, lw - (i // 3))
        draw.line([(x1, y1), (x2, y2)], fill=color, width=line_w)

def draw_corner_bracket(draw, x, y, corner, size=90, lw=3):
    """
    Draw L-shaped corner bracket.
    corner: 'tl', 'tr', 'bl', 'br'
    """
    s = size

```

```

c = GOLD_BRIGHT

if corner == 'tl':
    draw.line([(x, y), (x + s, y)], fill=c, width=lw)
    draw.line([(x, y), (x, y + s)], fill=c, width=lw)
elif corner == 'tr':
    draw.line([(x, y), (x - s, y)], fill=c, width=lw)
    draw.line([(x, y), (x, y + s)], fill=c, width=lw)
elif corner == 'bl':
    draw.line([(x, y), (x + s, y)], fill=c, width=lw)
    draw.line([(x, y), (x, y - s)], fill=c, width=lw)
elif corner == 'br':
    draw.line([(x, y), (x - s, y)], fill=c, width=lw)
    draw.line([(x, y), (x, y - s)], fill=c, width=lw)

def draw_border_rect(draw, margin, lw=3):
    """Draw thin gold border rect inset from edges."""
    m = margin
    w, h = PAGE_W, PAGE_H
    draw.rectangle([m, m, w - m, h - m], outline=GOLD_BRIGHT, width=lw)

def draw_bottom_scrip(canvas, scrim_h_frac=0.18):
    """Draw gradient scrim at bottom for text readability."""
    scrim_h = int(PAGE_H * scrim_h_frac)
    scrim = Image.new('RGBA', (PAGE_W, scrim_h), (0, 0, 0, 0))
    scrim_draw = ImageDraw.Draw(scrim)

    for y in range(scrim_h):
        alpha = int(200 * (y / scrim_h) ** 0.6)
        scrim_draw.rectangle([0, y, PAGE_W, y + 1], fill=(0, 0, 0, alpha))

    canvas.paste(scrim, (0, PAGE_H - scrim_h), scrim)

def draw_top_scrip(canvas, scrim_h_frac=0.09):
    """Light top scrim for header text."""
    scrim_h = int(PAGE_H * scrim_h_frac)
    scrim = Image.new('RGBA', (PAGE_W, scrim_h), (0, 0, 0, 0))
    scrim_draw = ImageDraw.Draw(scrim)

    for y in range(scrim_h):
        alpha = int(160 * (1 - y / scrim_h) ** 0.5)
        scrim_draw.rectangle([0, y, PAGE_W, y + 1], fill=(0, 0, 0, alpha))

    canvas.paste(scrim, (0, 0), scrim)

def center_text(draw, text, font, y, color=None):
    color = color or GOLD_BRIGHT

```

```

bbox = draw.textbbox((0, 0), text, font=font)
tw = bbox[2] - bbox[0]
x = (PAGE_W - tw) // 2
draw.text((x, y), text, font=font, fill=color)
return bbox[3] - bbox[1] # return height

def text_width(draw, text, font):
    bbox = draw.textbbox((0, 0), text, font=font)
    return bbox[2] - bbox[0]

# — PAGE BUILDERS —————

BORDER_MARGIN = 54 # ~18mm inset for border rect at 300dpi equiv
BRACKET_INSET = 80 # where corner brackets sit (inset from edge)
BRACKET_SIZE = 110 # length of bracket arms

def apply_framing_overlay(canvas, draw, page_config):
    """
    Apply the Dragon Seed framing system:
    - Thin gold border rect
    - Four corner L-brackets
    - Diagonal multi-line clusters (top-right and bottom-left)
    - Optional FGE logo mark bottom-left
    - Text layer
    """
    # Border rect
    draw_border_rect(draw, BORDER_MARGIN, lw=3)

    # Corner brackets (inside the border rect)
    bi = BRACKET_INSET
    bs = BRACKET_SIZE
    draw_corner_bracket(draw, bi, bi, 'tl', bs, lw=3)
    draw_corner_bracket(draw, PAGE_W - bi, bi, 'tr', bs, lw=3)
    draw_corner_bracket(draw, bi, PAGE_H - bi, 'bl', bs, lw=3)
    draw_corner_bracket(draw, PAGE_W - bi, PAGE_H - bi, 'br', bs, lw=3)

    # Top-right diagonal cluster (lines go down-left from top-right corner zone)
    # Dragon Seed has them emanating from top-right, sweeping down at ~225 degree
    tr_x = PAGE_W - 160
    tr_y = 160
    draw_diagonal_lines_cluster(draw, tr_x, tr_y, 'tr',
                                angle_deg=225, # down-left
                                n_lines=7, spacing=26, length=600, lw=2)

    # Bottom-left diagonal cluster (lines go up-right from bottom-left corner zon
    bl_x = 160
    bl_y = PAGE_H - 160

```

```

draw_diagonal_lines_cluster(draw, bl_x, bl_y, 'bl',
                             angle_deg=45, # up-right
                             n_lines=7, spacing=26, length=600, lw=2)

# Now draw text from config
cfg = page_config

# Header area: top scrim + header text
if cfg.get('header'):
    draw_top_scrim(canvas)
    header_font = load_font(FONT_BOLD, 52)
    header_y = 110
    center_text(draw, cfg['header'], header_font, header_y, GOLD_BRIGHT)

# Bottom scrim + text
if cfg.get('footer_main') or cfg.get('footer_sub'):
    draw_bottom_scrim(canvas)

    if cfg.get('footer_main'):
        f_font = load_font(FONT_BOLD, 62)
        center_text(draw, cfg['footer_main'], f_font, PAGE_H - 420, GOLD_BRIG

    if cfg.get('footer_sub'):
        s_font = load_font(FONT_REG, 40)
        center_text(draw, cfg['footer_sub'], s_font, PAGE_H - 340, WHITE_DIM)

    if cfg.get('footer_tiny'):
        t_font = load_font(FONT_SERIF_I, 36)
        center_text(draw, cfg['footer_tiny'], t_font, PAGE_H - 270, GOLD_DIM)

# FGE mark bottom-left (inside border)
fge_font = load_font(FONT_BOLD, 38)
draw.text((BORDER_MARGIN + 30, PAGE_H - BORDER_MARGIN - 60),
          "FGE", font=fge_font, fill=GOLD_DIM)

# Edition mark bottom-right
if cfg.get('edition'):
    ed_font = load_font(FONT_REG, 36)
    ed_text = cfg['edition']
    ed_w = text_width(draw, ed_text, ed_font)
    draw.text((PAGE_W - BORDER_MARGIN - 30 - ed_w, PAGE_H - BORDER_MARGIN - 6
              ed_text, font=ed_font, fill=GOLD_DIM)

def build_page(photo_path, page_config, output_name, is_landscape_source=False):
    """Build a single full-bleed lookbook page."""
    print(f"Building {output_name}...")

```

```

photo = Image.open(photo_path).convert('RGB')

# Fill page with cover crop
bg = fit_cover(photo, PAGE_W, PAGE_H)

# Convert to RGBA for compositing
canvas = bg.convert('RGBA')
draw = ImageDraw.Draw(canvas)

apply_framing_overlay(canvas, draw, page_config)

# Convert back to RGB for PDF
final = canvas.convert('RGB')
final.save(f"{OUTPUT_DIR}/{output_name}", 'JPEG', quality=95, dpi=(300, 300))
print(f" -> saved {final.size}")
return final

# ——— DEFINE ALL PAGES —————

pages = [
    {
        'photo': '/home/claude/isolde_extracted-000.jpg',
        'output': '00_cover.jpg',
        'config': {
            'header': 'FERAL GLOSS EMPIRE • GENESIS DROP',
            'footer_main': 'ISOLDE VOSS',
            'footer_sub': 'ROOKIE CARD • 001 / 777',
            'footer_tiny': 'THE ANCHOR IS THE CHARACTER. ONCE LOCKED, IT DOES NO',
            'edition': 'FGE-SB-332',
        }
    },
    {
        'photo': '/home/claude/isolde_extracted-001.jpg',
        'output': '01_lookbook.jpg',
        'config': {
            'header': 'LOOKBOOK 01 • ISOLDE VOSS',
            'footer_main': None,
            'footer_sub': 'FERAL GLOSS EMPIRE • THE ANCHOR METHOD • IDENTITY',
            'footer_tiny': None,
            'edition': None,
        }
    },
    {
        'photo': '/home/claude/isolde_extracted-002.jpg',
        'output': '02_lookbook.jpg',

```

```

        'config': {
            'header': 'LOOKBOOK 02 • ISOLDE VOSS',
            'footer_main': None,
            'footer_sub': 'FERAL GLOSS EMPIRE • THE ANCHOR METHOD • IDENTITY',
            'footer_tiny': None,
            'edition': None,
        }
    },
    {
        'photo': '/home/claude/isolde_extracted-003.jpg',
        'output': '03_lookbook.jpg',
        'config': {
            'header': 'LOOKBOOK 03 • ISOLDE VOSS',
            'footer_main': None,
            'footer_sub': 'FERAL GLOSS EMPIRE • THE ANCHOR METHOD • IDENTITY',
            'footer_tiny': None,
            'edition': None,
        }
    },
    {
        'photo': '/home/claude/isolde_extracted-004.jpg',
        'output': '04_lookbook.jpg',
        'config': {
            'header': 'LOOKBOOK 04 • ISOLDE VOSS',
            'footer_main': None,
            'footer_sub': 'FERAL GLOSS EMPIRE • THE ANCHOR METHOD • IDENTITY',
            'footer_tiny': None,
            'edition': None,
        }
    },
]

```

— BUILD ALL PAGES —————

```

built_pages = []
for p in pages:
    result = build_page(p['photo'], p['config'], p['output'])
    built_pages.append(result)

```

— ASSEMBLE PDF —————

```

print("\nAssembling PDF...")
from reportlab.lib.pagesizes import landscape
from reportlab.pdfgen import canvas as rl_canvas

pdf_path = f"{OUTPUT_DIR}/Isolde_Voss_Lookbook_FullBleed.pdf"

```

Use PIL to save as multi-page PDF directly

```
built_pages[0].save(  
    pdf_path,  
    save_all=True,  
    append_images=built_pages[1:],  
    resolution=300  
)  
print(f"PDF assembled: {pdf_path}")  
print("Done.")
```